🔴 **PROGRAM 1: Maven → Gradle**

📍 **WHERE TO WORK**

C:\Users\walee\eclipse-workspace\result-app

🪜 **PART A: CREATE MAVEN PROJECT**

✅ **Step 1: Create Project (Eclipse)**

👉 File → New → Maven Project
👉 Enter:

groupId: pgm1
artifactId: result-app

🪜 **PART B: ADD CODE**

✅ **Step 2: Result.java**

```
package pgm1;

public class Result {
   public static String display(int m) {
      if(m<0 || m>100) return "invalid";
      else if(m<40) return "fail";
      else return "pass";
   }

   public static void main(String[] args) {
      System.out.println(display(55));
   }
}
```

✅ **Step 3: ResultTest.java**

```
package pgm1;

import org.testng.Assert;
import org.testng.annotations.Test;

public class ResultTest {

 @Test
```

```
 public void testPass() {
  Assert.assertEquals(Result.display(55), "pass");
 }

 @Test
 public void testFail() {
  Assert.assertEquals(Result.display(35), "fail");
 }

 @Test
 public void testInvalid() {
  Assert.assertEquals(Result.display(-5), "invalid");
 }
}
```

✅ Step 4: Add dependency (pom.xml)

```
<dependency>
 <groupId>org.testng</groupId>
 <artifactId>testng</artifactId>
 <version>7.5.1</version>
 <scope>test</scope>
</dependency>
```

🪜 PART C: RUN MAVEN

✅ Step 5: Open Terminal

```
cd C:\Users\walee\eclipse-workspace\result-app
```

✅ Step 6: Compile

```
mvn clean compile
```

✅ Step 7: Run Tests

```
mvn test
```

✅ Output

```
BUILD SUCCESS
Tests run: 3, Failures: 0
```

🪜 PART D: RUN PROGRAM

✅ Step 8:

java -cp target/classes pgm1.Result

✅ Output

pass

🪜 PART E: CONVERT TO GRADLE

✅ Step 9: Install Gradle

👉 Download:

https://gradle.org/releases/

👉 Extract to:

C:\Gradle

👉 Add to PATH:

C:\Gradle\bin

✅ Step 10: Verify

gradle -v

✅ Step 11: Initialize Gradle

gradle init

👉 Select:

yes → Maven conversion
Groovy DSL → 2
new APIs → yes

✅ Step 12: Build

gradle build

✅ **Output**

BUILD SUCCESSFUL

🔴 **PROGRAM 2: Continuous Integration with Jenkins**

📍 **WHERE TO WORK**

- Project:

C:\Users\walee\eclipse-workspace\result-app

- Jenkins:

http://localhost:8080

🪜 **PART A: PUSH PROJECT TO GITHUB**

✅ **Step 1: Open Git Bash**

cd C:\Users\walee\eclipse-workspace\result-app

✅ **Step 2: Initialize repo**

```
git init
git add .
git commit -m "initial commit"
```

✅ **Step 3: Connect GitHub**

```
git branch -M main
git remote add origin https://github.com/your-username/your-repo.git
git push -u origin main
```

🪜 **PART B: INSTALL & CONFIGURE JENKINS**

✅ **Step 4: Install Jenkins**

👉 Download:

https://www.jenkins.io/download/

👉 Install .msi

✅ Step 5: Open Jenkins

http://localhost:8080

👉 Unlock using password:

C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword

✅ Step 6: Install Plugins

Install:

Git, Maven, Pipeline, TestNG

✅ Step 7: Configure Tools

👉 Go:

Manage Jenkins → Global Tool Configuration

Add:

JDK → JDK17
Maven → Maven3 (Install automatically)

🪜 PART C: CREATE JOB

✅ Step 8: Create Job

New Item → Freestyle Project → maven-ci-job

✅ Step 9: Configure Git

Repository URL:
https://github.com/your-username/your-repo.git

✅ Step 10: Build Step

👉 Add:

Invoke top-level Maven targets

👉 Enter:

clean test

👉 Select:

Maven Version → Maven3

🪜 PART D: RUN BUILD

✅ Step 11: Run

Click → Build Now

✅ Step 12: Check Output

👉 Open build console

✓ Output:

BUILD SUCCESS
Tests run: 3, Failures: 0

🪜 PART E: TESTNG REPORT

✅ Step 13: Add Post-build Action

Publish TestNG Results

👉 Path:

**/testng-results.xml

✅ Step 14: Run again

✓ See TestNG report in Jenkins

🎯 OPTIONAL: AUTO BUILD (Poll SCM)

✅ Step 15:

Configure → Build Triggers → Poll SCM

Add:

H/2 * * * *

🔴 PROGRAM 3: Docker (TO THE POINT)

📍 WHERE TO WORK

C:\Users\walee\OneDrive\Documents\Code\lab3

🪜 PART A: SINGLE CONTAINER

✅ Step 1: Create files

📄 app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Dockerized Flask App!"

app.run(host='0.0.0.0', port=5000)
```

📄 requirements.txt

```
flask
```

📄 Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

✅ Step 2: Open terminal

```
cd C:\Users\walee\OneDrive\Documents\Code\lab3
```

✅ Step 3: Build image

```
docker build -t flask-app .
```

✅ **Step 4: Run container**

docker run -p 5000:5000 flask-app

✅ **Step 5: Output**

👉 Open:

http://localhost:5000

✓ Output:

Hello from Dockerized Flask App!

🪜 **PART B: PUSH TO DOCKER HUB**

✅ **Step 6: Login**

docker login

✅ **Step 7: Tag**

docker tag flask-app waleed4480/flask-app

✅ **Step 8: Push**

docker push waleed4480/flask-app

🪜 **PART C: MULTI-CONTAINER**

📁 **Structure**

```
lab3-multi/
 app1/
 app2/
 docker-compose.yml
```

✅ **Step 9: app1 code**

```
from flask import Flask
app = Flask(__name__)
```

```
@app.route('/')
def hello():
    return "Hello from App 1!"

app.run(host='0.0.0.0', port=5000)
```

✅ **Step 10: app2 code**

```
import requests
response = requests.get("http://app1:5000/")
print("Response from App 1:", response.text)
```

✅ **Step 11: docker-compose.yml**

```
version: "3.9"

services:
  app1:
    build: ./app1
    ports:
      - "5050:5000"

  app2:
    build: ./app2
    depends_on:
      - app1
```

✅ **Step 12: Run**

```
docker-compose up --build
```

✅ **Step 13: Output**

👉 Browser:

http://localhost:5050

✓ Output:

Hello from App 1!

👉 Terminal:

Response from App 1: Hello from App 1!

🔴 **PROGRAM 4: Kubernetes (TO THE POINT)**

📍 **WHERE TO RUN**

👉 Open **CMD / PowerShell**

(Optional)

cd C:\Users\walee\OneDrive\Documents\Code\lab4

🪜 **PART A: START CLUSTER**

✅ **Step 1: Start Minikube**

minikube start --driver=docker

⏳ Wait 2–5 minutes

✅ **Step 2: Verify**

kubectl get nodes

✓ Output:

minikube Ready

🪜 **PART B: DEPLOY NGINX**

✅ **Step 3: Create Pod**

kubectl run nginx-pod --image=nginx

✅ **Step 4: Check Pod**

kubectl get pods

✓ Output:

nginx-pod Running

✅ **Step 5: Expose Pod**

kubectl expose pod nginx-pod --type=NodePort --port=80

✅ **Step 6: Open App**

minikube service nginx-pod

✓ Browser → NGINX page

🪜 **PART C: DEPLOY FLASK APP**

✅ **Step 7: Create Deployment**

👉 If you have your image:

kubectl create deployment flask-deploy --image=waleed4480/flask-app

👉 OR use nginx (safe):

kubectl create deployment flask-deploy --image=nginx

✅ **Step 8: Check**

kubectl get deployments
kubectl get pods

✅ **Step 9: Expose**

kubectl expose deployment flask-deploy --type=NodePort --port=5000

✅ **Step 10: Open**

minikube service flask-deploy

✓ Output (if Flask):

Hello from Dockerized Flask App!

🪜 **PART D: SCALE**

✅ **Step 11: Scale**

kubectl scale deployment flask-deploy --replicas=3

✅ **Step 12: Verify**

kubectl get pods

✓ Output:

3 pods running

🧹 **OPTIONAL (CLEANUP)**

kubectl delete deployment flask-deploy
kubectl delete pod nginx-pod
minikube stop

🔴 **PROGRAM 5: Terraform (AWS EC2)**

📍 **WHERE TO WORK**

D:\terraform

🪜 **PART A: SETUP**

✅ **Step 1: Install Terraform**

👉 Download:

https://developer.hashicorp.com/terraform/downloads

👉 Extract to:

C:\terraform

👉 Add to PATH:

C:\terraform

✅ **Step 2: Verify**

terraform -v

✓ Shows version

🪜 **PART B: CREATE CONFIG FILE**

✅ **Step 3: Create folder**

D:\terraform

✅ Step 4: Create file

main.tf

✅ Step 5: Add code

```
provider "aws" {
  region = "us-west-2"
}

resource "aws_instance" "ec2_machine" {
  ami          = "ami-07b0c09aab6e66ee9"
  instance_type = "t2.micro"

  tags = {
    Name = "Terra EC2"
  }
}
```

🪜 PART C: RUN TERRAFORM

✅ Step 6: Open terminal

cd D:\terraform

✅ Step 7: Initialize

terraform init

✓ Downloads AWS provider

✅ Step 8: Plan

terraform plan

✓ Shows what will be created

✅ Step 9: Apply

terraform apply

👉 Type:

yes

✓ Creates EC2 instance

🪜 **PART D: VERIFY**

✅ **Step 10:**

👉 Go to:

AWS Console → EC2 → Instances

✓ You will see:

Terra EC2 running

🪜 **PART E: DESTROY**

✅ **Step 11:**

terraform destroy

👉 Type:

yes

✓ Deletes instance